

Безопасность данных (2)

- Введение
- Настройка параметров PHP
- Безопасная загрузка файлов
- Что важно
- Что дальше

Введение

В предыдущем теме мы рассмотрели вопросы безопасной обработки **символьных данных, полученных от пользователя**. Рассмотрели вопросы фильтрации входных данных. Выяснили, что такое санитизация и валидация данных на стороне сервера. Надеюсь, вопрос, а что же выполнять в первую очередь - санитизацию или валидацию на повестке дня не стоит.

Рассмотрим не менее важный вопрос безопасной обработки данных, полученных от пользователя. Загрузка двоичных (бинарных) файлов, стандартной возможностью **пользовательских форм**.

Примечание. Загружать файлы можно разными способами. Один из часто используемых – **асинхронная загрузка**. Асинхронная загрузка будет кратко рассмотрена в теме: **PHP и технологии**.

Загрузка файлов является практически неотъемлемым атрибутом современного Web-приложения.

Тема является логическим продолжением темы **Загрузка файлов**, в случае необходимости повторите теоретический материал и примеры кодинга.

Настройка параметров PHP

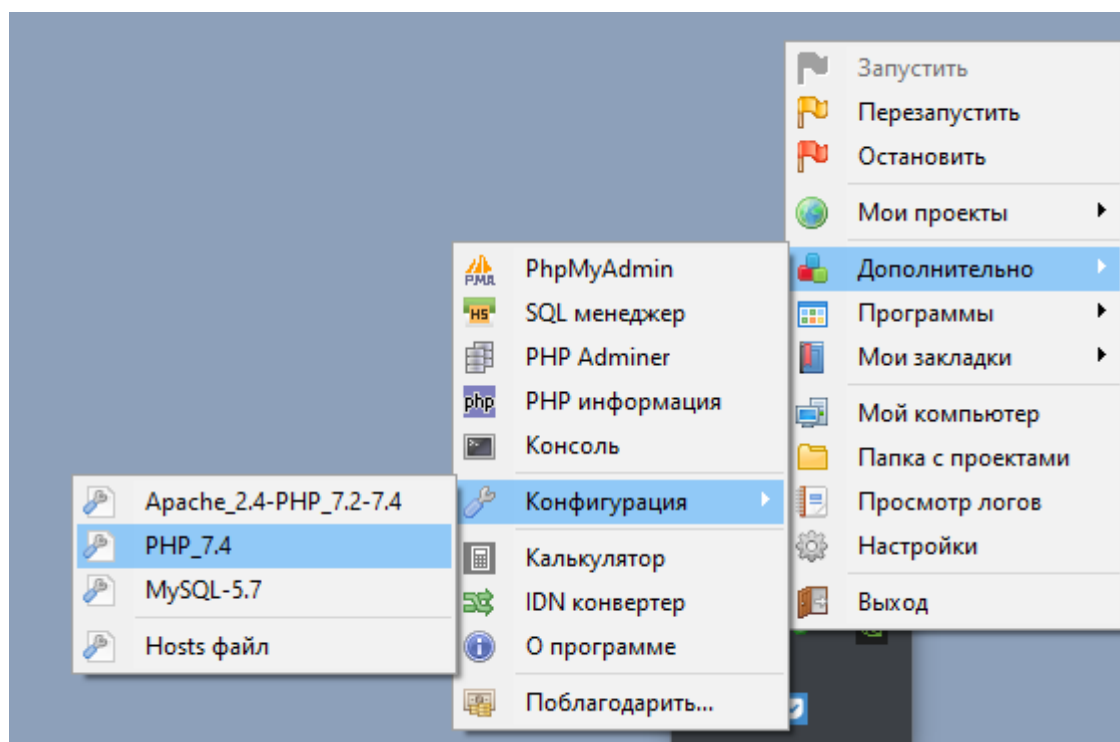
При загрузке может понадобиться возможность редактирования файла **php.ini**.

php.ini — это **конфигурационный** файл, содержащий **настройки интерпретатора PHP**. Синтаксис файла довольно прост, что позволяет редактировать его, не обладая глубокими специальными знаниями. Файл можно редактировать в любом **текстовом редакторе**.

Примечание. Не припомню, чтобы загрузка не выполнялась из-за проблем с настройками конфигурационного файла

Файл конфигурации считывается при запуске PHP. Местонахождение файла зависит от операционной системы, на которой работает сервер.

Я использую локальный сервер **OpenServer**, для просмотра конфигурационного файла нужно перейти в раздел Дополнительно -> Конфигурация -> PHP (ваша версия PHP).



Даже если проблемы не возникают, неплохо знать основные настройки **конфигурационного файла**, влияющие на загрузку файлов на сервер.

; Время выполнения скрипта в секундах (максимальное)

max_execution_time = 30

; Максимальное потребление памяти одним php скриптом

memory_limit = 128M

; Флаг загрузки файлов (On - загрузка разрешена)

file_uploads = On

; Максимально разрешённое количество одновременно загружаемых файлов

max_file_uploads = 20

; Максимально допустимый размер данных, отправляемых методом POST

post_max_size = 100M

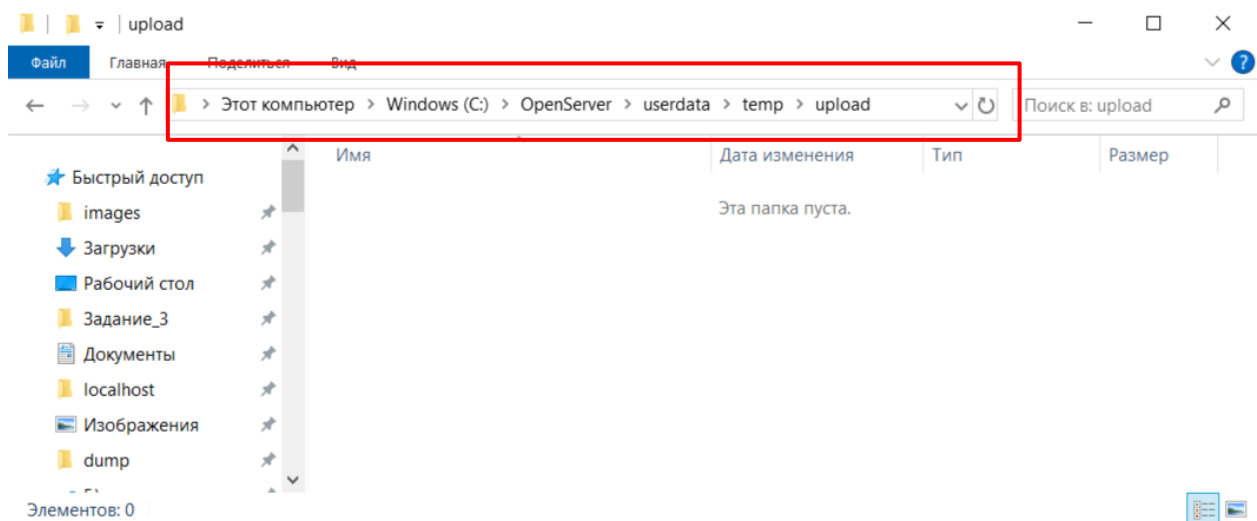
; Папка для временного хранения файлов во время загрузки, в OpenServer присутствует специальная переменная %sprogdir%

upload_tmp_dir = "%sprogdir%/userdata/temp/"

; Максимально допустимый размер загружаемого файла

upload_max_filesize = 5M

Директория временного хранения файлов на сервере OpenServer представлена на рисунке.



Возможные проблемы при загрузке:

- Загрузка файлов может быть отключена в настройках PHP директивой **file_uploads**.
- Не загружаются файлы большого размера, - причина в ограничениях конфигурационного файла.

К сведению. Конфигурационный файл **php.ini** необходимо настраивать согласно бизнес-логики создаваемого проекта.

Безопасная загрузка файлов

Проверка загружаемого файла

Следующая функция, используемая при загрузке двоичных файлов очень важная. Использовать ее необходимо каждый раз, когда планируете выполнять какие-то действия с загруженными файлами не связанные с перемещением файлов в директорию постоянного хранения.

Примечание. Обязательно ознакомьтесь с разделом урока "Что важно".

is_uploaded_file

is_uploaded_file () — определяет, был ли файл загружен при помощи HTTP POST.

Описание

```
is_uploaded_file($filename);
```

Возвращает **true**, если файл **filename** был загружен при помощи **HTTP POST**. Для правильной работы функции **is_uploaded_file()** нужен аргумент:

- **\$_FILES['userfile']['tmp_name']**.

Имя загруженного файла на клиентской машине:

- **\$_FILES['userfile']['name']** не подходит.

Параметры

- **filename** - имя проверяемого файла.

Возвращаемые значения

Возвращает **true** в случае успешного выполнения или **false** в случае возникновения ошибки.

Пример работы функции в коде листинга example_1.

Примечание. Подробный разбор **вариантов загрузки файлов** и обоснование необходимости применения функции **is_upload_file()** выходит за рамки текущего курса.

Загрузка изображений

Если ваш скрипт написан в расчете на то, что от пользователя придет файл изображения, необходимо проверить, а **является ли** полученный

файл изображением. Сделать это можно с помощью функции **getimagesize()**.

getimagesize

getimagesize () — получение размера изображения.

Описание

```
getimagesize($filename, $image_info);
```

Функция определит размер любого заданного, поддерживаемого изображения и вернёт этот размер вместе с типом файла и текстовой строкой **height / width**, которую можно будет использовать внутри тега HTML **img**, а также вернёт соответствующий **тип содержимого HTTP**.

Параметры

- **filename** – задаёт файл, сведения о котором будут извлекаться. Это может быть локальный или удалённый (зависит от конфигурации) файл.
- **image_info** – позволяет извлечь некоторые расширенные сведения из файла изображения.

Возвращаемые значения

Возвращает массив из 7 элементов.

- **Индексы 0 и 1** содержат ширину и высоту изображения. Некоторые форматы файлов могут хранить несколько изображений или не содержать изображения вообще. В таких случаях **getimagesize()** не сможет определить размер изображения. В таком случае **getimagesize()** вернёт нули в качестве значений высоты и ширины.
- **Индекс 2** содержит одну из констант типа изображения **IMAGETYPE_XXX**.

- **Индекс 3** содержит строку со значениями ширины и высоты изображения `height="yyy" width="xxx"`, которая может быть использована внутри **img** тега.
- **Mime** - соответствующий MIME-тип изображения.
- **Channels** принимает значение 3 для RGB картинок и 4 для CMYK.
- **Bits** - глубина цвета, число бит для каждого цвета.

В случае возникновения ошибки функция вернёт **false**.

example_1. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_1. server.php. Тестируем функцию getimagesize()

```
<?php

    // проверка, является ли файл результатом HTTP POST загрузки

    if (is_uploaded_file($_FILES["avatar"]["tmp_name"])) {

        // получим информацию о загружаемом файле

        $file = getimagesize($_FILES["avatar"]["tmp_name"]);

        // информация о файле, извлекаемая getimagesize

        echo "<pre>";

        var_dump($file);

        echo "</pre>";

    } else {

        // предупреждение об опасном действии пользователя

        echo "<h3>Файл не выбран или не загружается при помощи
```

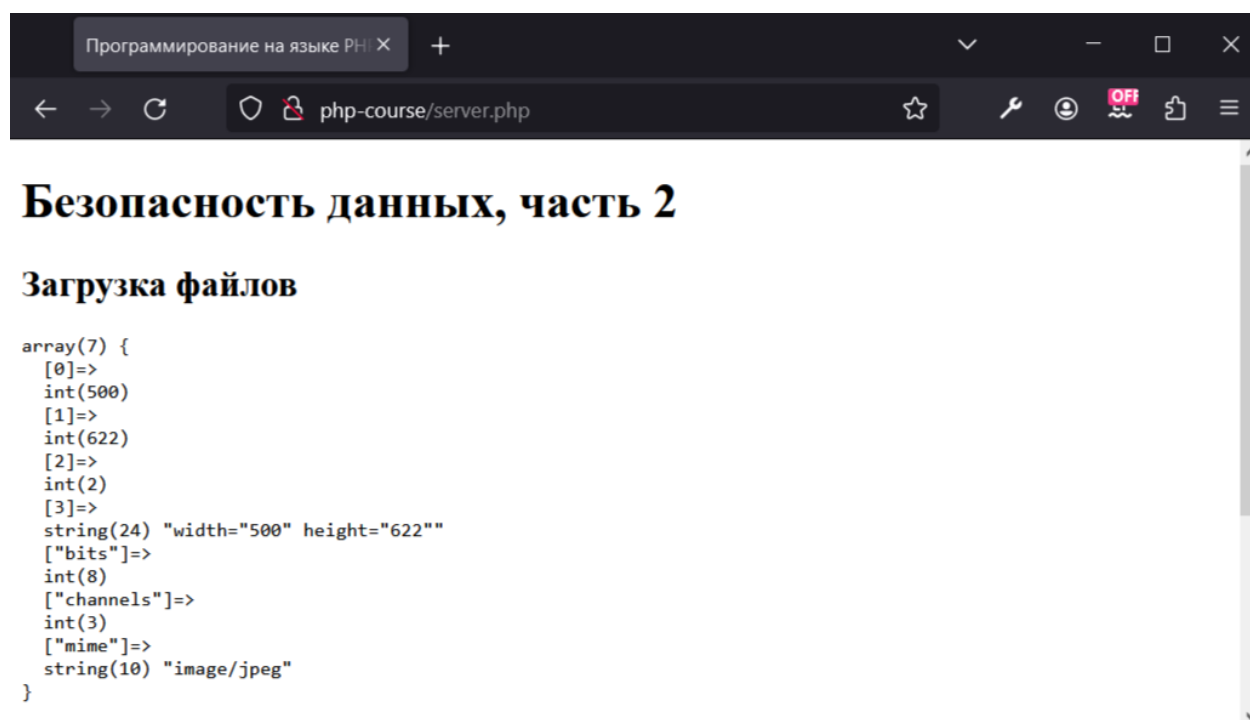
```
HTTP POST</h3>" ;

}

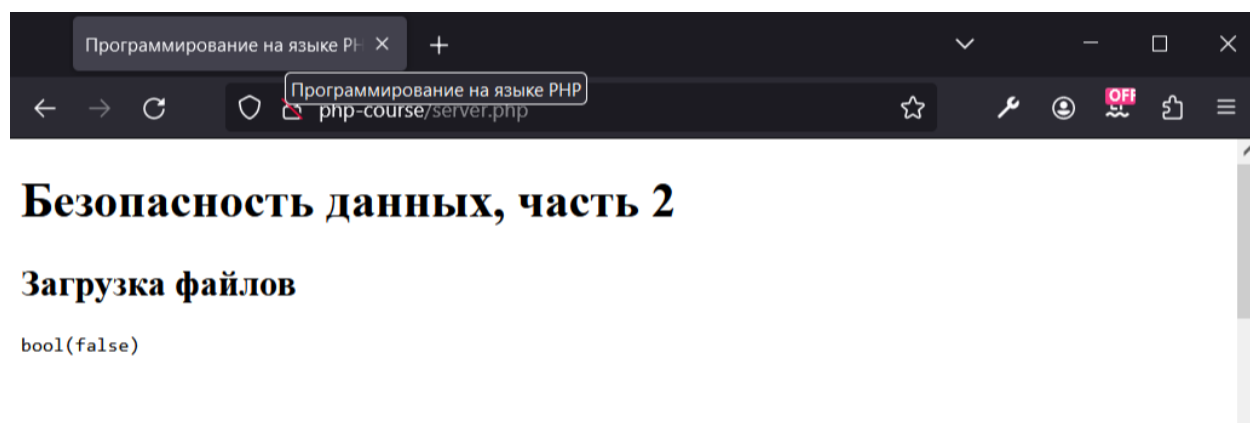
?>
```

Протестируйте код листинга **example_1**. Попробуйте загрузить файлы разных типов, отследите вывод функции **var_dump()**.

На следующих скриншотах представления ситуация загрузки файла изображения.



И файла текстового формата.



В демонстрационном примере **example_2** приведу пример **шаблона сценария** обработки графического файла аватара на сервере.

example_2. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_2. Шаблон сценария с проверкой загрузки

```
<?php

$_ERROR_LOADING = [];

// проверим загрузку на наличие ошибок

if ($_FILES['avatar']['error'] !== UPLOAD_ERR_OK ||
!is_uploaded_file($_FILES["avatar"]["tmp_name"])) {

    // если есть ошибка: $_FILES['avatar']['error']

    // или файл был загружен не через HTTP POST:
    is_uploaded_file($_FILES["avatar"]["tmp_name"])

    // вернём ошибку

    $_ERROR_LOADING[] = "Ошибка загрузки файла";

}

else {

    // проверка, действительно ли отправляемый файл
    изображение

    if (getimagesize($_FILES["avatar"]["tmp_name"])){

        // если файл действительно изображение,

        // выполняем над ним какие-то действия
```

```

    }

    else {

        // если файл не изображение

        $_ERROR_LOADING[] = "Загружаемый файл не является
        файлом изображения";

    }

}

// если массив ошибок не пуст, выводим их в браузер

if (count($_ERROR_LOADING)) {

    echo "<pre>";

    print_r($_ERROR_LOADING);

    echo "</pre>";

} else {

    // выводим сообщение об успешной загрузке файла
    изображения

    echo "<h3>Файл аватара 'загружен' на сервер</h3>";

}

?>

```

Полный код сценария загрузки изображения аватара представлен в коде примера **example_3**. Будьте внимательны, в форме примера используется скрытое текстовое поле **MAX_FILE_SIZE**, ограничивающее размер загружаемого файла.

example_3. index.html

```

<form action="server.php" enctype="multipart/form-data"
method="post">

    <!-- поле MAX_FILE_SIZE определяет максимальный размер
    загружаемого файла -->

```

```
<input type="hidden" name="MAX_FILE_SIZE" value="300000" />

Загрузите ваш аватар: <input type="file" name="avatar"><p>

<input type="submit" name="load" value="Загрузить файл">

</form>
```

example_3. server.php

```
<?php

$_ERROR_LOADING = [];

// проверим загрузку на наличие ошибок

if ($_FILES['avatar']['error'] != UPLOAD_ERR_OK) {

    // если при загрузке произошла ошибка, запомним
    информацию о ней

    switch ($_FILES['avatar']['error']) {

        case UPLOAD_ERR_INI_SIZE:

            $_ERROR_LOADING[] = "Размер принятого файла
            превысил максимально допустимый размер, который
            задан директивой upload_max_filesize
            конфигурационного файла php.ini (код ошибки: 1)";

            break;

        case UPLOAD_ERR_FORM_SIZE:

            $_ERROR_LOADING[] = "Размер загружаемого файла
            превысил значение MAX_FILE_SIZE, указанное в
            HTML-форме (код ошибки: 2)";

            break;

        case UPLOAD_ERR_PARTIAL:

            $_ERROR_LOADING[] = "Загружаемый файл был получен
            только частично (код ошибки: 3)";

            break;
```

```
case UPLOAD_ERR_NO_FILE:

    $_ERROR_LOADING[] = "Файл не был загружен (код
    ошибки: 4) ";

}

}

else {

    // проверка, действительно ли отправляемый файл
    изображение

    if (getimagesize($_FILES["avatar"]["tmp_name"])) {

        // если файл действительно изображение

        // текущее расположение файла

        $current_path = $_FILES['avatar']['tmp_name'];

        // оригинальное имя файла

        $filename = $_FILES['avatar']['name'];

        // место постоянного хранения файла

        $new_path = __DIR__ . '/upload/' . $filename;

        // перемещение загруженного файла

        if (!move_uploaded_file($current_path, $new_path)) {

            // если во время перемещения возникла ошибка

            $_ERROR_LOADING[] = "Не удалось переместить файл
            в директорию хранения";

        }

    }

    else {

        $_ERROR_LOADING[] = "Загружаемый файл не является
        файлом изображения";

    }

}
```

```
// если массив ошибок не пуст, выводим их в браузер

if (count($_ERROR_LOADING)) {

    echo "<pre>";

    print_r($_ERROR_LOADING);

    echo "</pre>";

}

else {

    // выводим сообщение об успешной загрузке файла
    изображения

    echo "<h3>Файл аватара успешно загружен на сервер</h3>";

    // выводим изображение аватарки

    echo "<img src='upload/" . $filename . "'
width='250px'>";

}

?>
```

Обратите внимание, в коде не используется функция **is_uploaded_file**

Примечание. В официальных документах есть предупреждение: "Не используйте `getimagesize()` для проверки того, что файл является изображением". Как вариант, можно использовать функцию **`exif_imagetype()`**, которая, к тому же, работает намного быстрее (если для вас это вообще важно).

Функция **`getimagesize()`** ожидает, что **filename** является корректным **файлом изображения**. Если будет передан файл не содержащий изображение, он может быть **некорректно интерпретирован** как изображение и функция завершится успешно, но массив при этом будет содержать **бессмысленные значения**.

exif_imagetype

exif_imagetype () — определяет тип изображения.

Описание

```
exif_imagetype($file);
```

Exif_imagetype() считывает начальные байты изображения и проверяет их сигнатуру.

Параметры

- **file** – изображение, тип которого требуется определить.

Возвращаемые значения

Если корректная сигнатура обнаружена, функция вернёт соответствующую типу изображения **константу**. В противном случае функция вернёт **false**.

Возвращаемое значение то же, что и во **втором аргументе** при возврате из функции **getimagesize()**, однако **exif_imagetype()** **значительно быстрее**.

Значение	Константа
1	IMAGETYPE_GIF
2	IMAGETYPE_JPEG
3	IMAGETYPE_PNG
4	IMAGETYPE_SWF
5	IMAGETYPE_PSD
6	IMAGETYPE_BMP

* - полный набор констант можно посмотреть в открытых источниках

Рассмотрим пример использования функции **exif_imagetype()**.

example_4. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_4. server.php. Тестируем функцию exif_imagetype()

```
<?php

    if (!empty($_FILES["avatar"]["tmp_name"])) {

        // если файл выбран, проверим его тип

        $type = [

            1 => 'IMAGETYPE_GIF',

            2 => 'IMAGETYPE_JPEG',

            3 => 'IMAGETYPE_PNG',

            4 => 'IMAGETYPE_SWF',

            5 => 'IMAGETYPE_PSD',

            6 => 'IMAGETYPE_BMP'

        ];

        $num = exif_imagetype($_FILES["avatar"]["tmp_name"]);

        echo 'Попытка загрузить файл типа: ' . $type[$num];

    } else {

        // иначе файл не выбран

        echo "<h3>Файл не выбран</h3>";

    }

?>
```

Код загрузки файла с проверкой функцией **exif_imagetype()** представлен в следующем примере.

example_5. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_5. server.php. Загрузка с проверкой exif_imagetype()

```
<?php

    if (!empty($_FILES["avatar"]["tmp_name"])) {

        echo "Функция <b>exif_imagetype</b> возвратила: ";

        var_dump(exif_imagetype($_FILES["avatar"]["tmp_name"]));

        // проверка, действительно ли отправляемый файл -
        изображение

        if (exif_imagetype($_FILES["avatar"]["tmp_name"])) {

            // файл - изображение, можно с ним работать

            echo "<h3>Добро пожаловать, будем сотрудничать
            ...</h3>";

        } else {

            // файл - не изображение,

            // дальнейшие действия зависят от алгоритма сценария

            echo "<h3>Загружаемый файл не является файлом
            изображения</h3>";

        }

    } else
```



```
echo "<h3>Выберите файл для загрузки</h3>";  
?>
```

Немного усложним задачу. В демонстрационном примере **example_6** проверим файл на предмет принадлежности к изображению, но при этом разрешим сохранять файлы изображений только **некоторых типов**.

example_6. index.html

```
<form action="server.php" enctype="multipart/form-data"  
method="post">  
  
    Загрузите ваш аватар: <input type="file" name="avatar"><p>  
  
    <input type="submit" name="load" value="Загрузить файл">  
  
</form>
```

example_6. server.php. Загрузка некоторых типов изображений

```
<?php  
  
    // файл для загрузки нужно выбрать,  
  
    // если файл не будет выбран, появится ошибка  
  
    // разрешенные форматы графических файлов для загрузки  
  
    define ('ALLOW_FILE_EXT', array(IMAGETYPE_GIF,  
    IMAGETYPE_JPEG, IMAGETYPE_PNG));  
  
    // проверка, есть ли в массиве разрешенных типов загружаемый  
    файл  
  
    if (!in_array(exif_imagetype($_FILES["avatar"]["tmp_name"]),  
    ALLOW_FILE_EXT)) {  
  
        // файл может быть изображением, но при этом не входить в
```

```
        список разрешенных

        echo "Загружаемый файл не относится к списку

        разрешенных";

    } else {

        echo "Загружаемый файл будет сохранен на сервере";

    }

?>
```

Загрузка файлов произвольных типов

Загружать на сервер можно не только файлы изображений. Список разрешенных к загрузке файлов может быть достаточно широк.

Предположим, что в следующем примере мы должны принимать файлы только с расширением **.jpg** и **.pdf**.

Обратите внимание, в коде не используется функция **is_uploaded_file**

example_7. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    <!-- поле MAX_FILE_SIZE определяет максимальный размер
    загружаемого файла -->

    <input type="hidden" name="MAX_FILE_SIZE" value="300000" />

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_7. server.php. Загрузка с проверкой расширения файла

```
<?php

// иницилируем массив сбора сообщений об ошибках

$_ERROR_LOADING = [];

// проверим загрузку на наличие ошибок

if ($_FILES['avatar']['error'] !== UPLOAD_ERR_OK) {

    // если при загрузке произошла ошибка, заппомним
    информацию о ней в массив

    switch ($_FILES['avatar']['error']) {

        case UPLOAD_ERR_INI_SIZE:

            $_ERROR_LOADING[] = "Размер принятого файла
            превысил максимально допустимый размер, который
            задан директивой upload_max_filesize
            конфигурационного файла php.ini (код ошибки: 1)";

            break;

        case UPLOAD_ERR_FORM_SIZE:

            $_ERROR_LOADING[] = "Размер загружаемого файла
            превысил значение MAX_FILE_SIZE, указанное в
            HTML-форме (код ошибки: 2)";

            break;

        case UPLOAD_ERR_PARTIAL:

            $_ERROR_LOADING[] = "Загружаемый файл был получен
            только частично (код ошибки: 3)";

            break;

        case UPLOAD_ERR_NO_FILE:

            $_ERROR_LOADING[] = "Файл не был загружен (код
            ошибки: 4)";

    }

}
```

```

else {

    $ext = pathinfo($_FILES['avatar']['name'],
    PATHINFO_EXTENSION);

    // разрешение расширения файлов для загрузки

    define ('ALLOW_FILE_EXT', array('jpg', 'pdf'));

    // если загружаемый файл 'jpg' или 'pdf'

    if (in_array($ext, ALLOW_FILE_EXT)) {

        // текущее расположение файла

        $current_path = $_FILES['avatar']['tmp_name'];

        // оригинальное имя файла

        $filename = $_FILES['avatar']['name'];

        // место постоянного хранения файла

        $new_path = __DIR__ . '/upload/' . $filename;

        // перемещение загруженного файла

        if (!move_uploaded_file($current_path, $new_path)) {

            // если во время перемещения возникла ошибка

            $_ERROR_LOADING[] = "Не удалось переместить файл
            в директорию хранения";

        }

    } else {

        $_ERROR_LOADING[] = "Файлы с расширением {$ext} не
        разрешаются к загрузке";

    }

}

// если массив ошибок не пуст

if (count($_ERROR_LOADING)) {

    echo "<pre>";

    print_r($_ERROR_LOADING);
}

```

```
    } else {  
  
        // выводим сообщение об успешной загрузке файла  
  
        echo "<h3>Файл успешно загружен на сервер</h3>";  
  
    }  
  
?>
```

Тестируя пример из **example_7** можно обнаружить интересный факт. Попробуйте загрузить на сервер файл с расширением отличным от разрешенных (например **.png**). Не получится, формат **.png** не разрешен к загрузке. Поменяйте расширение файла, например на **.jpg** и получите **беспроблемную загрузку**.

Дело в том, что тип файла определяется по **заголовку браузера**, а браузер мы обманули поменяв расширение.

Примечание. Пользователю и браузеру доверять нельзя. Тип файла необходимо проверять самостоятельно.

Протестирую сказанное простым примером. Меняйте расширение одного файла определенного типа, следите за выводом функции **print_r()**. Браузер очень доверчив.

example_8. index.html

```
<form action="server.php" enctype="multipart/form-data"  
method="post">  
  
    Загрузите ваш аватар: <input type="file" name="avatar"><p>  
  
    <input type="submit" name="load" value="Загрузить файл">  
  
</form>
```

example_8. server.php. Определяем тип файла по заголовку браузера

```
<?php
```

```
echo "<pre>";

print_r($_FILES);

echo "</pre>";

?>
```

Глобальный массив `$_FILES` содержит всю информацию о загруженных файлах. `$_FILES['userfile']['type']` содержит **mime**-тип файла, в случае, если браузер предоставил такую информацию. Этот **mime-type** **не проверяется на стороне PHP**, так что не полагайтесь на его значение без проверки.

Загрузка файлов определенных mime-type

Обозначенная проблема не велика. Перестанем доверять браузеру, и проверим тип файла самостоятельно.

`$_FILES['userfile']['type']` возвращает тип загруженного файла или **mime-type**, если точнее. Для файла **jpg** этот элемент будет иметь значение **image/jpeg**. Если загрузить текст, то элемент получит значение **text/plain**. Для разных типов файлов разным будет и **mime-type**.

Ниже приведены некоторые распространенные **mime**-типы:

- JPEG: **image/jpeg**;
- PNG: **image/png**;
- Текст: **text/plain**;
- Word: **application/msword**.

mime_content_type

mime_content_type () — определяет MIME-тип содержимого файла.

Описание

```
mime_content_type($file);
```

Возвращает MIME-тип содержимого файла.

Список параметров

- **file** – путь к проверяемому файлу.

Возвращаемые значения

Возвращает тип содержимого в формате MIME, например **text/plain** или **application/octet-stream** или **false** в случае возникновения ошибки.

Ошибки

В случае неудачного завершения работы генерируется ошибка уровня E_WARNING - обычные предупреждения (не критичные ошибки).

В демонстрационном примере **example_9** тестируем функцию **mime_content_type()**. Экспериментируйте.

example_9. index.html

```
<form action="server.php" enctype="multipart/form-data"
method="post">

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>
```

example_9. server.php. Определение типа файла по его mime-type

```
<?php

    // если файл отправлен из формы

    if (!empty($_FILES["avatar"]["tmp_name"])) {

        // то что сообщил браузер
```

```

    echo "<h3>То, что сообщил браузер</h3>";

    echo "<pre>";

    print_r($_FILES);

    echo "</pre>";

    // реальный mime-type файла

    echo "<h3>То, что определила функция
mime_content_type</h3>";

    echo mime_content_type($_FILES["avatar"]["tmp_name"]);

}

else {

    echo "<h3>Файл не выбран</h3>";

}

?>

```

Полный код загрузки файлов определенных **mime**-type представлен в коде следующего примера. Будьте внимательны, в форме примера используется поле **MAX_FILE_SIZE**, ограничивающее размер загружаемого файла.

example_10. index.html

```

<form action="server.php" enctype="multipart/form-data"
method="post">

    <!-- поле MAX_FILE_SIZE определяет максимальный размер
загружаемого файла -->

    <input type="hidden" name="MAX_FILE_SIZE" value="300000" />

    Загрузите ваш аватар: <input type="file" name="avatar"><p>

    <input type="submit" name="load" value="Загрузить файл">

</form>

```


example_10. server.php. Безопасный код загрузки файлов

```
<?php

$_ERROR_LOADING = [];

// проверим загрузку на наличие ошибок

if ($_FILES['avatar']['error'] !== UPLOAD_ERR_OK) {

    // если при загрузке произошла ошибка, запомним
    информацию о ней

    switch ($_FILES['avatar']['error']) {

        case UPLOAD_ERR_INI_SIZE:

            $_ERROR_LOADING[] = "Размер принятого файла
            превысил максимально допустимый размер, который
            задан директивой upload_max_filesize
            конфигурационного файла php.ini (код ошибки: 1)";

            break;

        case UPLOAD_ERR_FORM_SIZE:

            $_ERROR_LOADING[] = "Размер загружаемого файла
            превысил значение MAX_FILE_SIZE, указанное в
            HTML-форме (код ошибки: 2)";

            break;

        case UPLOAD_ERR_PARTIAL:

            $_ERROR_LOADING[] = "Загружаемый файл был получен
            только частично (код ошибки: 3)";

            break;

        case UPLOAD_ERR_NO_FILE:

            $_ERROR_LOADING[] = "Файл не был загружен (код
            ошибки: 4)";

    }

}

else {
```

```
$mime_type =  
mime_content_type($_FILES["avatar"]["tmp_name"]);  
  
// разрешение расширения файлов для загрузки  
  
define ('ALLOW_MIME_TYPE', array('image/jpeg',  
'image/png'));  
  
if (in_array($mime_type, ALLOW_MIME_TYPE)) {  
    // текущее расположение файла  
  
    $current_path = $_FILES['avatar']['tmp_name'];  
  
    // оригинальное имя файла  
  
    $filename = $_FILES['avatar']['name'];  
  
    // место постоянного хранения файла  
  
    $new_path = __DIR__ . '/upload/' . $filename;  
  
    // перемещение загруженного файла  
  
    if (!move_uploaded_file($current_path, $new_path)) {  
        // если во время перемещения возникла ошибка  
  
        $_ERROR_LOADING[] = "Не удалось переместить файл  
        в директорию хранения";  
  
    }  
  
} else {  
  
    $_ERROR_LOADING[] = "Файлы с типом {$mime_type} не  
    допускаются к загрузке";  
  
}  
  
}  
  
// выведем список ошибок если таковые имеются  
  
if (count($_ERROR_LOADING)) {  
  
    echo "<h3>Список возникших ошибок</h3>";  
  
    echo "<pre>";  
  
    print_r($_ERROR_LOADING);  
}
```

```
    } else {  
  
        // выводим сообщение об успешной загрузке файла  
        изображения  
  
        echo "<h3>Файл аватара успешно загружен на сервер</h3>";  
  
        // выводим изображение аватарки  
  
        echo "<img src='upload/" . $filename . "'  
        width='250px'>";  
  
    }  
  
?>
```

И опять обратите внимание, в коде не используется функция **is_uploaded_file**. Я написал, что функция важная, но, практически нигде ее не использовал. Разбираемся.

Что важно

Функция **is_uploaded_file** выполняет проверку, был ли файл загружен при помощи HTTP POST. В каких-то сценариях используется, в каких-то нет. На самом деле все достаточно просто.

Ровно такую же проверку выполняет функция **перемещения файла** в директорию постоянного хранения **move_uploaded_file**. Отсюда следует вывод номер один.

Важно. Использовать функцию **is_uploaded_file** в паре с **move_uploaded_file** не имеет смысла.

Когда использовать? Принятый файл далеко не всегда загружается в директорию сервера, он может быть переправлен по почте или записан в базу данных. В таком случае функция **move_uploaded_file** **использоваться не будет**. Отсюда следует вывод номер два.

Важно. Если действия, выполняемые с файлом, не предполагают его перемещение с помощью функции **move_uploaded_file**, использование функции **is_uploaded_file** обязательно.

Что дальше

Закономерный вопрос. Ну, отправили данные на сервер. Там их проверили, почистили, вывели. Какой в этом смысл?

Все просто. Или не очень. А точнее так - алгоритм действий простой, реализация не всегда.

Важно. В подавляющем количестве случаев следующий шаг - **запись принятых данных в базу данных.**